

Pokedex Lite

De React a React Native, sin dar nada por sentado — construyendo una app real que consume la PokeAPI.

Instructor/Docente: Edwin Rozo Gómez

7-10 h

TypeScript desde cero

Plan A: emulador · Plan B: web

Este curso asume que ya sabes React (componentes, props, estado, hooks) pero **nunca has tocado TypeScript ni React Native**. Cada paso trae la explicación de “qué es esto y por qué lo hago”, no solo el comando a copiar.

¿DÓNDE CONFIGURO TODO (PLAN A Y PLAN B)?

Todo el setup de entorno —emulador Android y modo web— se hace en un único lugar: el **Módulo 2**. Es el único módulo donde instalas cosas; después de terminarlo, no vuelves a tocar configuración en el resto del curso.

0. Qué vas a construir

Una app de tres pantallas (tres rutas):

RUTA	PANTALLA	QUÉ HACE
/	Lista	Trae los primeros N Pokémon de la PokeAPI y los muestra en una lista
/pokemon/[name]	Detalle	Al tocar un Pokémon, muestra su imagen, tipos y estadísticas
/favoritos (opcional, módulo 11)	Favoritos	Lista los Pokémon marcados como favoritos, guardados localmente

0.1 Así se ve al terminar

Una vista rápida de las tres pantallas, lado a lado (lista → detalle → favoritos):

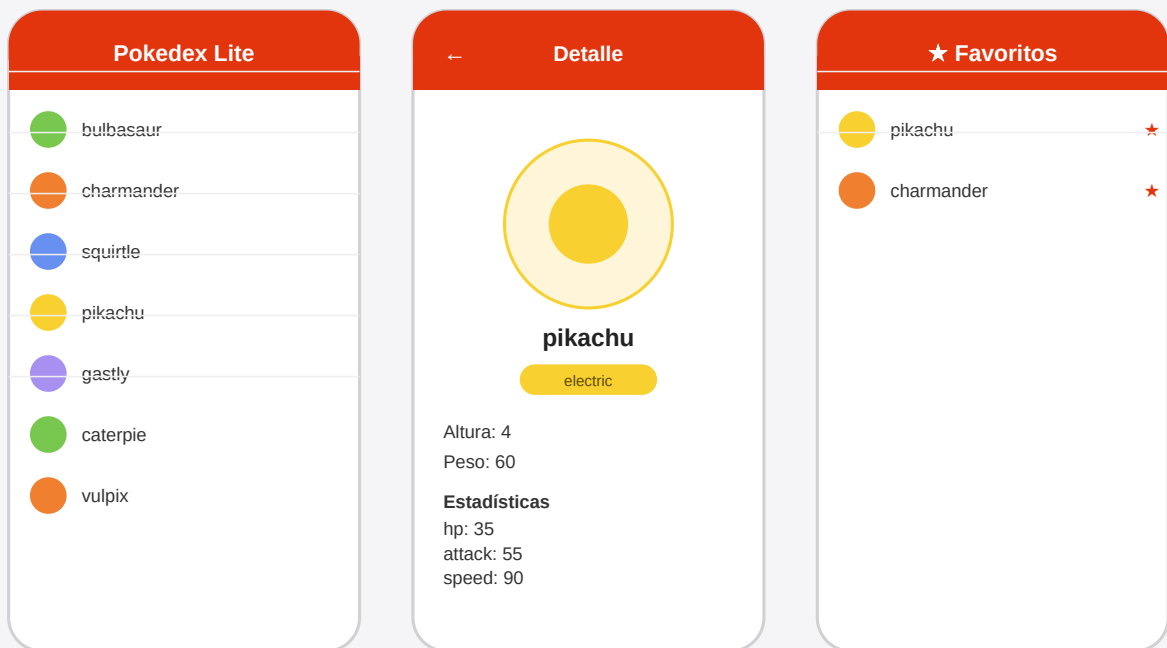


FIG. 1 – LISTA → DETALLE → FAVORITOS

1. TypeScript esencial, paso a paso

1.1 Qué es TypeScript y por qué te sirve

TypeScript **no es un lenguaje aparte** de JavaScript: es JavaScript con una capa extra llamada "tipos". Todo el JavaScript que ya sabes sigue funcionando igual. Lo único que cambia es que ahora puedes decirle al editor "esta variable siempre va a ser un texto" o "esta función siempre recibe un número", y el editor te avisa **mientras escribes** si algo no cuadra, antes de que la app se rompa en el celular.

Analogía: es como si React te avisara "estás pasando `undefined` a un prop que la pantalla necesita para no crashear" en el momento en que escribes el código, no cuando el usuario abre la app y ve una pantalla en blanco.

Dos cosas importantes para tranquilidad:

- Los tipos **desaparecen** al compilar — en tiempo de ejecución es JavaScript puro, tal cual lo conoces.
- Puedes ir muy despacio: TypeScript no te obliga a tipar absolutamente todo, en la mayoría de los casos **infiere** el tipo solo con lo que escribes.

1.2 Extensiones de archivo: `.ts` vs `.tsx`

- `.ts` — archivos de solo lógica (funciones, hooks, tipos), sin JSX.
- `.tsx` — archivos que además tienen JSX (`<View>...</View>`), es decir, tus componentes/pantallas.

En este curso vas a ver ambos: los hooks van en `.ts`, las pantallas van en `.tsx`.

1.3 Práctica: tipar variables (sin instalar nada)

Antes de tocar el proyecto, prueba esto en typescriptlang.org/play (pega el código y mira los subrayados rojos si algo está mal):

```
let nombre: string = "Ash";
let nivel: number = 5;
let esNovato: boolean = true;
```

La parte después de los `:` es el **tipo**. Ahora borra el `: string` y deja `let nombre = "Ash";` — TypeScript sigue sabiendo que es un string, solo que lo **infirió** automáticamente. Por eso en la práctica vas a escribir menos anotaciones de las que imaginas: solo las pones donde TS no puede adivinar solo.

Prueba a propósito un error, para que veas cómo se siente:

```
let nivel: number = 5;
nivel = "cinco"; //  el editor se queja aquí, antes de correr nada
```

Ese subrayado rojo **es** la razón de ser de TypeScript: errores atrapados en el editor, no en producción.

1.4 Funciones tipadas: declaración vs función flecha

Hay dos formas de escribir una función en JavaScript/TypeScript, y en este curso vamos a usar **siempre la segunda**, así que vale la pena que entiendas bien la diferencia antes de ver una sola línea más de código.

Forma 1 — declaración de función (la que probablemente ya conoces de React):

```
function saludar(nombre: string): string {  
  return `Hola, ${nombre}`;  
}
```

Forma 2 — función flecha (`arrow function`), asignada a una constante:

```
const saludar = (nombre: string): string => {  
  return `Hola, ${nombre}`;  
};
```

Ambas hacen exactamente lo mismo. Leyendo la segunda de izquierda a derecha: `const saludar` — estamos creando una constante llamada `saludar`; `= (nombre: string)` — le asignamos una función que recibe un parámetro `nombre` de tipo `string`; `: string` — esa función retorna un `string`; `=> { ... }` — la "flecha" separa los parámetros del cuerpo de la función.

Detalles que no vamos a dar por sentado:

- El `;` al final es porque, a diferencia de una declaración de función, esto es una **asignación** (le estás dando un valor a una constante), y las asignaciones en JS/TS terminan en punto y coma, igual que `const x = 5;`.
- Si el cuerpo de la función es una sola expresión que se retorna directo, puedes omitir las llaves `{ }` y el `return` — se llama **retorno implícito**: `ts const doble = (n: number): number => n * 2;` `// equivale exactamente a: const doble = (n: number): number => { return n * 2; };` En este curso vamos a usar llaves y `return` explícito casi siempre (es más fácil de leer cuando hay varias líneas), y solo usaremos el retorno implícito en funciones muy cortas de una línea.
- Si la función no recibe parámetros, los paréntesis van vacíos igual: `const saludar = (): string => { ... }.`
- ¿Por qué elegimos funciones flecha para todo el curso (componentes, hooks, funciones auxiliares) en vez de declaraciones de función? Por dos razones prácticas: es la convención más extendida hoy en proyectos React/React Native con TypeScript, y combina mejor con el patrón de exports que vas a usar en cada archivo (siguiente sección). No hay una razón "obligatoria" de TypeScript detrás — es una elección de estilo consistente que vamos a mantener en todo el proyecto.

Pruébalo tú mismo:

```
saludar("Ash"); //   
saludar(42);    //  error: 42 no es un string
```

El comportamiento frente a TypeScript es idéntico en ambas formas: `nombre: string` sigue siendo el tipo del parámetro, y `: string` después del paréntesis de parámetros sigue siendo el tipo de retorno (opcional, TS también lo infiere, pero es buena costumbre ponerlo en funciones que no son triviales).

1.5 Exports: nombrado vs por defecto (y la convención que usa este curso)

Ya sabes crear una función. Ahora, ¿cómo hace otro archivo para usarla? Ahí entra `export`. Hay dos maneras:

Export nombrado — se declara justo al lado de la función, con la palabra `export` delante:

```
export const saludar = (nombre: string): string => {  
  return `Hola, ${nombre}`;  
};
```

Para usarla en otro archivo, la importas por su nombre exacto, entre llaves:

```
import { saludar } from "../saludos";
```

Export por defecto — se declara al final del archivo, separado de la función, y solo puede haber **uno por archivo**:

```
const saludar = (nombre: string): string => {  
  return `Hola, ${nombre}`;  
};  
  
export default saludar;
```

Para usarla en otro archivo, la importas sin llaves, y puedes ponerle **cualquier nombre** al importarla (lo cual, en la práctica, genera inconsistencias entre archivos):

```
import saludo from "../saludos"; // el nombre "saludo" es arbitrario, nadie te obliga a  
llamarlo igual
```

La convención que seguimos en este curso, sin excepciones salvo una: todas las funciones que escribamos —componentes, hooks, funciones auxiliares— se declaran como función flecha con `export` justo al lado, es decir `export const NombreDeLaFuncion = (...) => { ... }`. Esto tiene dos ventajas concretas para alguien empezando: el nombre con el que exportas es siempre el mismo con el que se importa (no hay ambigüedad), y tu editor puede ofrecerte autocompletado e ir-a-la-definición de forma más confiable.

La única excepción: los archivos de pantallas dentro de la carpeta `app/` (las rutas). Expo Router —la librería que arma la navegación a partir de los archivos, que vas a conocer en el módulo 4— exige que el componente de la pantalla sea el `export default` de ese archivo específico; así es como Expo Router sabe qué componente renderizar para esa ruta. En esos archivos vamos a escribir igual la función como

flecha (por consistencia), pero vamos a agregar `export default NombreDePantalla;` al final, en vez de poner `export` al lado de la declaración. Vas a ver el primer ejemplo concreto de esto en el módulo 4, con una explicación puntual ahí mismo.

Fuera de esos archivos de ruta, todo en este curso lleva `export const` al lado de la función. Ningún archivo de hook, tipo o componente reusable va a tener un `export default`.

1.6 Arreglos tipados

```
let tipos: string[] = ["electric", "flying"];
let niveles: number[] = [1, 2, 3];
```

`string[]` se lee "arreglo de strings". Si intentas `tipos.push(5)`, TS se queja porque `5` no es un string.

1.7 `interface` : la forma de un objeto

Esta es la herramienta que más vas a usar en el curso. Una `interface` describe **qué propiedades tiene un objeto y de qué tipo son**, nada más — no es una clase, no tiene comportamiento.

```
interface Pokemon {
  id: number;
  name: string;
  types: string[];
  isFavorite?: boolean; // el ? significa "esta propiedad es opcional"
}

const pikachu: Pokemon = { id: 25, name: "pikachu", types: ["electric"] };
// no puse isFavorite y no hay error, porque es opcional

const roto: Pokemon = { id: 25, name: "pikachu" };
// error: falta "types", que NO es opcional
```

Piensa en una `interface` como un molde: si tu objeto no calza con el molde (le falta una propiedad obligatoria, o tiene el tipo equivocado), TS te avisa antes de que ese objeto viaje por media app y falle en un lugar inesperado.

`interface` vs `type` : hacen cosas muy parecidas; en este curso usamos `interface` para "formas de datos" (como las respuestas de una API) porque es la convención más común en proyectos React/React Native. No necesitas saber la diferencia técnica todavía.

1.8 Tipar props de un componente

Esto reemplaza lo que en React "normal" quizá resolvías con `PropTypes` o simplemente sin nada. Aplicando ya la convención de los puntos 1.4 y 1.5 (función flecha + `export const` al lado):

```
interface PokemonCardProps {
  name: string;
  imageUrl: string;
  onPress: () => void; // una función que no recibe nada y no retorna nada
}

export const PokemonCard = ({ name, imageUrl, onPress }: PokemonCardProps) => {
  // acá dentro, "name" ya es string, "onPress" ya es función — con autocompletado incluido
};
```

Nota de sintaxis, porque aquí hay algo nuevo respecto al ejemplo de `saludar`: los parámetros de la función no son `nombre: string` sueltos, sino `{ name, imageUrl, onPress }`: `PokemonCardProps`. Eso es **destructuring** de un único objeto de parámetros (el objeto de props, tal como ya lo conoces de React) seguido de `: PokemonCardProps`, que le dice a TypeScript la forma completa de ese objeto de una sola vez, en vez de tipar cada campo por separado.

`() => void` es la forma de tipar "una función". `void` significa "no retorna nada útil". Si la función recibiera un argumento, se vería así: `onPress: (id: number) => void`.

`PokemonCard` no es un archivo de ruta (no vive dentro de `app/`), así que sigue la regla general: `export const` al lado, sin `export default`.

1.9 Tipar `useState` con genéricos (explicado sin jerga)

```
const [pokemons, setPokemons] = useState<Pokemon[]>([]);
const [loading, setLoading] = useState<boolean>(true);
const [error, setError] = useState<string | null>(null);
```

Ese `<Pokemon[]>` entre los signos de menor/mayor que se llama **genérico**. No necesitas entender la teoría — en la práctica significa: "le estoy diciendo a `useState` qué tipo de dato va a guardar". Sin él, TS a veces no puede adivinar (por ejemplo, `useState([])` no sabe si ese arreglo vacío va a llenarse de números, strings u objetos `Pokemon`; con `useState<Pokemon[]>([])` se lo dices explícitamente).

Regla práctica: cuando el valor inicial ya deja claro el tipo (`useState(true)`, `useState("")`), no hace falta el genérico, TS infiere solo. Cuando el valor inicial es ambiguo (arreglo vacío `[]`, o `null` que después va a convertirse en un objeto), sí conviene ponerlo.

1.10 Tipos que vas a ver seguido (glosario rápido)

- `string | null` — "unión de tipos": puede ser un string, o puede no haber nada todavía. Muy común para representar "esto aún no cargó".
- `any` — evítalo casi siempre; le dice a TS "no revises nada aquí" y pierdes toda la ayuda del editor.
- `unknown` — como `any` pero seguro: te obliga a verificar el tipo antes de usarlo. Lo vas a ver al capturar errores (`catch (err)`, `err` es `unknown`).
- `as` — un "cásting" manual (`data as Pokemon`), le dice a TS "confía en mí, sé que esto es un `Pokemon`". Úsalo solo cuando estás seguro; es una vía de escape, no la regla general.

- **?. (optional chaining)** — `pokemon?.name` no explota si `pokemon` es `null`, simplemente da `undefined`.
- **?? (nullish coalescing)** — `nombre ?? "sin nombre"` usa el valor de la izquierda, salvo que sea `null` / `undefined`, en cuyo caso usa el de la derecha.

1.11 Async/await con tipos (lo que vas a usar al pedir datos a la API)

```
export const obtenerPokemon = async (nombre: string): Promise<Pokemon> => {
  const res = await fetch(`https://pokeapi.co/api/v2/pokemon/${nombre}`);
  const data: Pokemon = await res.json();
  return data;
};
```

Sintaxis nueva aquí: la palabra `async` ahora va **antes** de los paréntesis de parámetros, no antes de `function` como en la forma declarativa (`async function`). El resto se lee igual que cualquier función flecha: recibe `nombre: string`, y retorna `Promise<Pokemon>` — es decir, "esta función es asíncrona y, cuando termine, va a entregar un `Pokemon`". Verás este patrón calcado, con `export const` al lado tal como acordamos en 1.5, en los hooks del módulo 7.

1.12 El `tsconfig.json` : no lo toques (por ahora)

Expo genera automáticamente un `tsconfig.json` con `strict: true` — la configuración recomendada para principiantes, porque te obliga a manejar los casos `null` / `undefined` en vez de dejar que exploten más adelante. No necesitas editarlo para este curso.

1.13 Autoevaluación rápida

Antes de seguir, deberías poder responder esto sin mirar atrás (si no, vuelve a leer la sección correspondiente):

1. ¿Qué diferencia hay entre `.ts` y `.tsx`?
2. ¿Qué diferencia real de comportamiento hay entre una declaración de función y una función flecha, para lo que vamos a hacer en este curso?
3. ¿Por qué en este curso usamos `export const Nombre = () => {}` en vez de `export default function Nombre() {}` en casi todos los archivos?
4. ¿Cuál es la única excepción a esa regla, y por qué existe?
5. ¿Qué significa el `?` después del nombre de una propiedad en una `interface`?
6. ¿Por qué `useState<Pokemon[]>([])` lleva el genérico y `useState(true)` no lo necesita?
7. ¿Qué hace `string | null` distinto a solo `string`?

Con esto tienes todo el TypeScript que necesitas para el resto del curso — el resto lo vas a ver aplicado directamente en código real, que es la mejor forma de que se quede.

2. Configuración del entorno — Plan A y Plan B (paso único)

Este es el **único módulo del curso donde instalas software**. Al terminarlo vas a poder correr la app de dos formas: en un emulador Android (Plan A) y en el navegador (Plan B), sin volver a tocar configuración después.

2.1 Requisitos base (necesarios para ambos planes)

1. Instala **Node.js** en su versión LTS desde nodejs.org.
2. Verifica la instalación abriendo una terminal y corriendo: `bash node -v npm -v` Deberías ver un número de versión en cada comando, no un error.
3. Instala **VS Code** (o el editor de tu preferencia) — trae soporte de TypeScript integrado, no necesitas instalar nada extra para eso.

Con esto ya tienes lo mínimo para el Plan B. Sigue leyendo para el Plan A.

2.2 Plan A — Emulador Android vía Android Studio

Esta es la parte más larga del módulo, pero se hace **una sola vez en todo el curso**.

Paso 1 — Instalar el JDK 17 (lo necesita el sistema de build de Android):

- macOS: `brew install --cask zulu17`, luego agrega `JAVA_HOME` a tu `~/.zshrc`.
- Windows: `choco install -y microsoft-openjdk17` (o el instalador oficial de Microsoft).
- Linux: instala OpenJDK 17 desde el gestor de paquetes de tu distribución.

Paso 2 — Instalar Android Studio: descárgalo de developer.android.com/studio y corre el instalador eligiendo el tipo "Standard". Esto instala Android Studio, el SDK de Android y el emulador.

Paso 3 — Verificar el SDK: dentro de Android Studio ve a **Settings → Languages & Frameworks → Android SDK**. Confirma que la última **SDK Platform** esté marcada, y en la pestaña "SDK Tools" confirma que **Android Emulator** y **Android SDK Build-Tools** estén instalados.

Paso 4 — Variables de entorno: necesitas que tu terminal sepa dónde está el SDK.

- macOS/Linux, agrega a tu archivo de shell (`~/.zshrc` o `~/.bashrc`): `bash export ANDROID_HOME=$HOME/Library/Android/sdk # ruta típica en macOS export PATH=$PATH:$ANDROID_HOME/platform-tools`
- Windows: define `ANDROID_HOME` desde el Panel de Control → Variables de entorno, y agrega `platform-tools` a tu `Path`.

Verifica que funcionó cerrando y abriendo la terminal, y corriendo:

```
adb --version
```

Si ves una versión (no "command not found"), quedó bien configurado.

Paso 5 — Crear un dispositivo virtual (AVD): en Android Studio, **More Actions → Virtual Device Manager → Create Virtual Device**. Elige un modelo (Pixel es buena opción), selecciona una imagen de sistema (descárgala si te la pide) y dale **Finish**.

Paso 6 — Encender el emulador: desde el Device Manager, presiona el botón ► junto al dispositivo que creaste. Espera a que termine de bootear (puede tardar uno o dos minutos la primera vez) — deberías ver la pantalla de inicio de Android, como un celular real.

Con el emulador prendido y corriendo, el Plan A ya está listo.

2.3 Plan B — Web

No requiere ninguna instalación adicional a lo del paso 2.1. Expo Router puede servir automáticamente una versión web del mismo proyecto — es tu red de seguridad si el emulador falla, si estás en una máquina donde no puedes instalar Android Studio, o simplemente si quieres iterar más rápido sin esperar al emulador.

2.4 Alternativa extra: tu celular con Expo Go

No es obligatoria, pero si tienes tu teléfono a mano, instala la app **Expo Go** (Play Store / App Store). Sirve como una tercera opción de prueba, más rápida de configurar que el emulador, y útil más adelante para ver cómo se comporta el diseño en una pantalla real (ver módulo 6).

2.5 Checklist antes de seguir

No avances al módulo 3 hasta poder marcar todo esto:

- ☐ `node -v` y `npm -v` responden con un número de versión.
- ☐ `adb --version` responde con un número de versión (Plan A).
- ☐ Tienes al menos un dispositivo virtual creado en Android Studio (Plan A).
- ☐ Sabes que para el Plan B no necesitas instalar nada más.

En el módulo 3, al crear el proyecto, vas a probar ambos planes juntos por primera vez para confirmar que todo quedó bien.

3. Crear el proyecto

```
npx create-expo-app@latest pokedex-lite
cd pokedex-lite
```

Qué hace este comando, paso a paso:

1. `npx` descarga y ejecuta una herramienta sin instalarla permanentemente en tu máquina — es la forma estándar de correr generadores de proyectos.
2. `create-expo-app@latest` es el generador oficial de proyectos Expo.
3. El CLI te va a preguntar por la versión de SDK de Expo — deja la que venga sugerida por defecto (la más reciente estable).
4. `pokedex-lite` es el nombre de la carpeta que se crea con todo el proyecto adentro.

La plantilla por defecto **ya viene con TypeScript y Expo Router configurados** — no eliges nada extra, ya está listo.

3.1 Qué archivos se generaron y para qué sirve cada uno

```
pokedex-lite/
├─ app/
│  └─ _layout.tsx    ← configura la navegación (el "enrutador" raíz)
│     └─ index.tsx   ← esta es la ruta "/"
├─ assets/           ← imágenes e íconos de la app
├─ tsconfig.json     ← configuración de TypeScript (no la toques todavía)
├─ app.json           ← configuración de la app (nombre, ícono, splash screen)
└─ package.json      ← dependencias y scripts del proyecto
```

Para arrancar con una base más limpia (sin las pantallas de ejemplo que trae la plantilla), corre:

```
npm run reset-project
```

Esto mueve el contenido de ejemplo a una carpeta `app-example/` (por si quieres consultarlo después) y te deja `app/index.tsx` minimalista, listo para trabajar desde cero.

3.2 Primera prueba — aquí confirmas Plan A y Plan B juntos

```
npx expo start
```

Este comando levanta el **servidor de desarrollo** (Metro): un proceso que empaqueta tu código y se lo sirve a donde sea que estés probando la app. Con el servidor corriendo, en esa misma terminal:

- Presiona `a` → abre la app en tu emulador Android (Plan A). Si el emulador no estaba prendido, Expo intenta abrirlo solo.

- Presiona [w](#) → abre la app en tu navegador (Plan B).
- Escanea el QR con la app **Expo Go** → la abre en tu celular físico.

Si ambos — [a](#) y [w](#) — te muestran la pantalla de ejemplo de Expo, la configuración del módulo 2 quedó perfecta y puedes avanzar tranquilo. Si algo falla, revisa el [módulo 10 de troubleshooting](#) antes de seguir.

4. Rutas con Expo Router (file-based routing)

La idea central de Expo Router: **cada archivo dentro de `app/` es una ruta**, igual que en Next.js. No hay que configurar un `Stack.Navigator` a mano llenándolo de pantallas — la carpeta ya es tu mapa de navegación.

Crea esta estructura (archivos nuevos dentro de `app/`):

```
app/
├─ _layout.tsx
├─ index.tsx           ← ruta "/" (lista)
├─ pokemon/
│  └─ [name].tsx       ← ruta "/pokemon/pikachu" (detalle, dinámica)
```

`app/_layout.tsx` define cómo se ven las transiciones entre pantallas. Usamos un `Stack` (pantallas que se "apilan" con flecha de volver arriba, el patrón de navegación más natural para este proyecto):

```
// app/_layout.tsx
import { Stack } from "expo-router";

const RootLayout = () => {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: "Pokedex Lite" }} />
      <Stack.Screen name="pokemon/[name]" options={{ title: "Detalle" }} />
    </Stack>
  );
};

export default RootLayout;
```

Aquí aparece por primera vez la excepción que adelantamos en 1.5: `app/_layout.tsx` vive dentro de la carpeta de rutas, así que Expo Router necesita encontrar un `export default` en este archivo para saber qué componente usar como layout raíz — si en cambio hubiéramos escrito `export const RootLayout = ...` sin el `export default` al final, Expo Router no encontraría ninguna pantalla para esa ruta y la app fallaría al arrancar. Por eso el patrón en **todo archivo dentro de `app/`** va a ser: declarar la función como flecha (misma sintaxis de siempre), y agregar una línea `export default NombreDeLaFuncion;` al final, en vez de poner `export` pegado a la declaración. Fuera de la carpeta `app/` —hooks, tipos, componentes reusables— seguimos con `export const` al lado, sin excepciones.

Cada `Stack.Screen` corresponde a un archivo dentro de `app/`, y `options={{ title: ... }}` es lo que aparece en la barra superior de esa pantalla.

El nombre del archivo `[name].tsx` (entre corchetes) es lo que hace la ruta **dinámica**: `name` queda disponible como parámetro dentro de la pantalla vía el hook `useLocalSearchParams`, que verás en el módulo 8.

Para navegar entre pantallas usas el componente `Link` (parecido a un `<a>` de la web) o el hook `useRouter` — ambos de `expo-router`, los verás en el módulo 8.

5. Estilos en React Native: StyleSheet y Flexbox

React Native **no usa CSS ni archivos** `.css` . No hay `className` , no hay cascada global, no hay selectores. En cambio, cada componente recibe un prop `style` con un objeto de JavaScript. Esta sección cubre lo que necesitas para no sentirte perdido.

5.1 Los componentes base (tu nuevo HTML)

WEB	REACT NATIVE	PARA QUÉ
<code><div></code>	<code><View></code>	Contenedor genérico
<code><p></code> / <code></code>	<code><Text></code>	Todo texto debe ir dentro de un <code><Text></code> — no puedes poner texto suelto dentro de un <code><View></code>
<code></code>	<code><Image></code>	Mostrar imágenes (necesita un objeto <code>source</code> , no un <code>src</code> string suelto)
<code><input></code>	<code><TextInput></code>	Campos de texto
<code><button></code> / elemento clickeable	<code><Pressable></code> (o <code><TouchableOpacity></code>)	Cualquier cosa tocable
—	<code><ScrollView></code> / <code><FlatList></code>	Contenido desplazable; <code>FlatList</code> está optimizado para listas largas

5.2 `StyleSheet.create` : tu reemplazo de CSS

```
import { View, Text, StyleSheet } from "react-native";

export const Saludo = () => {
  return (
    <View style={styles.contenedor}>
      <Text style={styles.titulo}>Hola</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  contenedor: { padding: 16, backgroundColor: "#fff" },
  titulo: { fontSize: 20, fontWeight: "bold", color: "#222" },
});
```

Nota: `styles` **no lleva `export`** — es un detalle de implementación de este archivo, ningún otro archivo necesita importarlo directamente (cada pantalla define su propio `styles`).

Puntos clave, comparado con CSS:

- Las propiedades van en **camelCase**: `backgroundColor` , no `background-color` .
- Los números **no llevan unidad** (`padding: 16` , no `"16px"`) — RN los interpreta como "dp" (density-independent pixels), una unidad que se ve consistente sin importar la densidad de píxeles del dispositivo.

- Los porcentajes sí van como string: `width: "100%"` .
- No hay shorthand de `border` como en CSS: tienes que poner `borderWidth` , `borderColor` y `borderStyle` por separado.
- `StyleSheet.create` no es obligatorio (podrías pasar un objeto plano `style={{ padding: 16 }}`), pero es la convención estándar: valida los nombres de propiedades con TypeScript (te avisa si escribes `colr` en vez de `color`) y es un poco más eficiente en tiempo de ejecución.
- **No hay cascada:** los estilos de un `View` padre no bajan a sus hijos (con la excepción de que un `<Text>` sí puede heredar estilo de texto de otro `<Text>` que lo envuelva). Cada componente necesita su propio `style` .

5.3 Flexbox: el sistema de layout por defecto

A diferencia de la web (donde `display: flex` hay que activarlo), en React Native todo `View` es flex por defecto. Dos diferencias clave con la web:

- `flexDirection` por defecto es `"column"` (apilado vertical), no `"row"` como en la web.
- No existen los `float` , `display: block/inline` , ni grid nativo — todo el layout se resuelve con flexbox.

Las tres propiedades que vas a usar el 90% del tiempo:

```
const styles = StyleSheet.create({
  fila: {
    flexDirection: "row",          // organiza los hijos en fila en vez de columna
    justifyContent: "space-between", // alineación en el eje principal (horizontal, porque
    es "row")
    alignItems: "center",         // alineación en el eje cruzado (vertical, porque es "row")
    gap: 8,                       // espacio entre hijos, sin necesitar márgenes manuales
  },
  contenedorCompleto: {
    flex: 1, // "ocupa todo el espacio disponible" — lo vas a ver en casi cada pantalla
  },
});
```

Truco mental: si `flexDirection` es `"row"` , piensa en `justifyContent` como "izquierda-derecha" y `alignItems` como "arriba-abajo". Si es `"column"` (el default), es al revés.

5.4 Estilos condicionales y combinados

`style` acepta un arreglo de estilos, no solo un objeto — muy útil para estados como "seleccionado" o "deshabilitado":

```
<View style={[styles.card, isFavorite && styles.cardFavorita]}>
```

Si `isFavorite` es `false` , ese valor se ignora (RN filtra los `false` / `null` / `undefined` del arreglo); si es `true` , se aplican ambos estilos, y el segundo pisa las propiedades que se repitan del primero — igual

que la cascada normal, pero explícita.

5.5 Aplicándolo a las pantallas del proyecto

Estos son los estilos que vas a usar en el módulo 8, ya anotados:

```
const styles = StyleSheet.create({
  center: {
    flex: 1, // ocupa toda la pantalla
    justifyContent: "center", // centra verticalmente (eje principal, column)
    alignItems: "center", // centra horizontalmente (eje cruzado, column)
  },
  list: { padding: 16 },
  row: {
    paddingVertical: 12,
    borderBottomWidth: 1, // "border" siempre desglosado en RN
    borderColor: "#eee",
  },
  name: { fontSize: 18, textTransform: "capitalize" },
});
```

5.6 Tipar estilos personalizados (opcional, pero útil)

Si en algún momento quieres que un componente propio reciba un `style` desde afuera (por ejemplo, para reusar `PokemonCard` en dos pantallas con look distinto), tipa el prop con `ViewStyle`, `TextStyle` o `ImageStyle`, importados de `"react-native"`:

```
import { ViewStyle } from "react-native";

interface PokemonCardProps {
  name: string;
  style?: ViewStyle; // opcional: quien use el componente puede pasar estilos extra
}
```

No lo necesitas para la versión base del proyecto, pero es el patrón a seguir cuando empieces a reusar componentes entre pantallas.

6. Diseño para distintos tamaños de pantalla (responsive)

React Native no tiene *media queries* como CSS. En su lugar, el diseño responsive se resuelve con **flexbox flexible** (evitar tamaños fijos) más, cuando hace falta, **JavaScript reactivo al tamaño de pantalla**.

6.1 Regla general: preferir flexible sobre fijo

En vez de fijar anchos en píxeles, usa `flex`, porcentajes, o `flexWrap`, para que el layout se adapte solo:

```
// ❌ evitar: ancho fijo que se ve mal en pantallas chicas o grandes
<View style={{ width: 320 }}>

// ✅ preferir: se adapta al contenedor
<View style={{ flex: 1 }}>
// o
<View style={{ width: "90%" }}>
```

6.2 `useWindowDimensions` : cuando sí necesitas saber el tamaño exacto

Para casos donde el layout tiene que cambiar de forma (no solo estirarse), usa este hook — es reactivo, así que si el usuario rota el celular o (en web) redimensiona la ventana, tu componente se vuelve a renderizar solo:

```
import { useWindowDimensions } from "react-native";

export const Pantalla = () => {
  const { width } = useWindowDimensions();
  const esAncho = width > 600; // umbral típico para "se ve como tablet/desktop"

  return (
    // usa "esAncho" para decidir, por ejemplo, cuántas columnas mostrar
  );
};
```

6.3 Aplicado al proyecto: lista en grilla según el ancho

En `app/index.tsx` (lo verás completo en el módulo 8), esto hace que la lista muestre más columnas en pantallas anchas (como el modo web en una ventana grande) y una sola columna en un celular angosto:

```
const { width } = useWindowDimensions();
const columnas = width > 600 ? 3 : width > 380 ? 2 : 1;

<FlatList
  key={columnas} // FlatList necesita remontarse si numColumns cambia
  data={pokemons}
  numColumns={columnas}
  // ...
/>
```

6.4 Zonas seguras (notch, barra de estado, gestos)

La plantilla por defecto de Expo Router ya maneja la zona segura automáticamente en las pantallas dentro de un `Stack` (el header respeta el notch/cámara). Si en algún extra armas una pantalla `sin` header (por ejemplo, un modal a pantalla completa), envuelve el contenido en `SafeAreaView` para no quedar debajo de la cámara o la barra de gestos:

```
import { SafeAreaView } from "react-native-safe-area-context";
// ya viene instalado por la plantilla de Expo, no necesitas agregarlo

<SafeAreaView style={{ flex: 1 }}>
  {/* contenido */}
</SafeAreaView>
```

6.5 Diferencias entre plataformas (iOS vs Android vs Web)

A veces un mismo valor se ve distinto entre plataformas (sombras, por ejemplo, se definen distinto en iOS y Android). Cuando lo necesites, `Platform.OS` te deja bifurcar:

```
import { Platform } from "react-native";

const styles = StyleSheet.create({
  card: {
    ...Platform.select({
      ios: { shadowColor: "#000", shadowOpacity: 0.1, shadowRadius: 4 },
      android: { elevation: 3 },
      web: { boxShadow: "0 1px 4px rgba(0,0,0,0.1)" },
    }),
  },
});
```

No lo necesitas para la versión mínima del proyecto — es una herramienta para cuando el pulido visual empiece a importar.

6.6 Cómo probar que el diseño responsive funciona

- **Emulador (Plan A):** prueba con más de un perfil de dispositivo (por ejemplo, un Pixel normal y una tablet) desde el Device Manager de Android Studio.

- **Web (Plan B):** simplemente achica y agranda la ventana del navegador — verás el `useWindowDimensions` reaccionar en vivo.
 - **Celular físico (Expo Go):** la prueba más realista para gestos táctiles y zonas seguras reales.
-

7. Tipar y consumir la PokeAPI

7.1 Los endpoints que vamos a usar

- Lista: `GET https://pokeapi.co/api/v2/pokemon?limit=20&offset=0`
- Detalle: `GET https://pokeapi.co/api/v2/pokemon/{nombre-o-id}`

No requieren API key ni headers especiales.

7.2 Define los tipos de las respuestas

Crea `types/pokemon.ts` :

```
// types/pokemon.ts
export interface PokemonListItem {
  name: string;
  url: string;
}

export interface PokemonListResponse {
  count: number;
  next: string | null;
  previous: string | null;
  results: PokemonListItem[];
}

export interface PokemonType {
  slot: number;
  type: { name: string; url: string };
}

export interface PokemonStat {
  base_stat: number;
  stat: { name: string };
}

export interface PokemonDetail {
  id: number;
  name: string;
  height: number;
  weight: number;
  sprites: {
    front_default: string | null;
  };
  types: PokemonType[];
  stats: PokemonStat[];
}
```

¿De dónde salen estos campos? De mirar la respuesta real de la API (puedes abrir <https://pokeapi.co/api/v2/pokemon/pikachu> directamente en el navegador y ver el JSON).

Escribir los tipos "a mano" mirando datos reales es exactamente lo que harías en un proyecto de trabajo antes de tener tipos compartidos con un backend propio.

7.3 Un hook reutilizable para pedir datos

Crea `hooks/usePokemonList.ts` :

```
// hooks/usePokemonList.ts
import { useEffect, useState } from "react";
import { PokemonListItem, PokemonListResponse } from "../types/pokemon";

export const usePokemonList = (limit: number = 20) => {
  const [pokemons, setPokemons] = useState<PokemonListItem[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;

    const fetchPokemons = async () => {
      try {
        setLoading(true);
        const res = await fetch(
          `https://pokeapi.co/api/v2/pokemon?limit=${limit}&offset=0`
        );
        if (!res.ok) throw new Error(`Error ${res.status}`);
        const data: PokemonListResponse = await res.json();
        if (!cancelled) setPokemons(data.results);
      } catch (err) {
        if (!cancelled) {
          setError(err instanceof Error ? err.message : "Error desconocido");
        }
      } finally {
        if (!cancelled) setLoading(false);
      }
    };

    fetchPokemons();
    return () => {
      cancelled = true;
    };
  }, [limit]);

  return { pokemons, loading, error };
};
```

Dos cosas nuevas en este archivo, con la lupa puesta:

- `usePokemonList` es la función principal del archivo: `export const` al lado, como manda la convención (no es un archivo de ruta).
- `fetchPokemons` , en cambio, es una función **interna**, que solo existe dentro de `usePokemonList` y no se usa desde ningún otro archivo — por eso es `const fetchPokemons = async () => { ... }` sin `export` : el `export` solo tiene sentido en la función que otros archivos van a importar.

Nota sobre `err instanceof Error` : dentro de un `catch` , TypeScript tipa `err` como `unknown` (repasa 1.10) — por eso hay que verificar de qué tipo es antes de leer `.message` .

Y `hooks/usePokemonDetail.ts` , muy parecido:

```
// hooks/usePokemonDetail.ts
import { useEffect, useState } from "react";
import { PokemonDetail } from "../types/pokemon";

export const usePokemonDetail = (name: string) => {
  const [pokemon, setPokemon] = useState<PokemonDetail | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;

    const fetchDetail = async () => {
      try {
        setLoading(true);
        const res = await fetch(`https://pokeapi.co/api/v2/pokemon/${name}`);
        if (!res.ok) throw new Error(`Error ${res.status}`);
        const data: PokemonDetail = await res.json();
        if (!cancelled) setPokemon(data);
      } catch (err) {
        if (!cancelled) {
          setError(err instanceof Error ? err.message : "Error desconocido");
        }
      } finally {
        if (!cancelled) setLoading(false);
      }
    };

    fetchDetail();
    return () => {
      cancelled = true;
    };
  }, [name]);

  return { pokemon, loading, error };
};
```

Separar el `fetch` en hooks (en vez de escribirlo directo en la pantalla) es una buena costumbre que ya conoces de React — aquí se traduce igual, solo que ahora todo viene tipado y como función flecha exportada al lado de su declaración.

8. Construir las pantallas

8.1 Lista — `app/index.tsx`

```

// app/index.tsx
import { FlatList, Text, View, StyleSheet, ActivityIndicator, Pressable, useWindowDimensions }
  from "react-native";
import { Link } from "expo-router";
import { usePokemonList } from "../hooks/usePokemonList";

const HomeScreen = () => {
  const { pokemons, loading, error } = usePokemonList(20);
  const { width } = useWindowDimensions();
  const columnas = width > 600 ? 3 : width > 380 ? 2 : 1;

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  if (error) {
    return (
      <View style={styles.center}>
        <Text>Ocurrió un error: {error}</Text>
      </View>
    );
  }

  return (
    <FlatList
      key={columnas}
      data={pokemons}
      numColumns={columnas}
      keyExtractor={(item) => item.name}
      contentContainerStyle={styles.list}
      renderItem={({ item }) => (
        <Link href={` /pokemon/${item.name}`} asChild>
          <Pressable style={styles.row}>
            <Text style={styles.name}>{item.name}</Text>
          </Pressable>
        </Link>
      )}
    />
  );
};

const styles = StyleSheet.create({
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  list: { padding: 16 },
  row: { flex: 1, paddingVertical: 12, paddingHorizontal: 8, borderBottomWidth: 1,
borderColor: "#eee" },
  name: { fontSize: 18, textTransform: "capitalize" },
});

export default HomeScreen;

```

Igual que en `app/_layout.tsx` (módulo 4): `app/index.tsx` es un archivo de ruta, así que `HomeScreen` se declara como función flecha (`const HomeScreen = () => { ... };`) y se exporta al final con `export default HomeScreen;` , en vez de `export const` al lado. `styles` , en cambio, no es una ruta ni algo que otro archivo necesite importar, así que se queda sin `export` , como constante interna del archivo.

`numColumns={columnas}` es lo que conecta esta pantalla con el módulo 6: en una ventana angosta ves una lista simple, en una pantalla ancha (tablet, o web con la ventana grande) se acomoda en varias columnas automáticamente.

8.2 Detalle — `app/pokemon/[name].tsx`

```
// app/pokemon/[name].tsx
import { View, Text, Image, StyleSheet, ActivityIndicator, useWindowDimensions } from
"react-native";
import { useLocalSearchParams } from "expo-router";
import { usePokemonDetail } from "../../hooks/usePokemonDetail";

const PokemonDetailScreen = () => {
  const { name } = useLocalSearchParams<{ name: string }>();
  const { pokemon, loading, error } = usePokemonDetail(name);
  const { width } = useWindowDimensions();
  const tamañoImagen = width > 600 ? 220 : 150; // imagen más grande en pantallas anchas

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  if (error || !pokemon) {
    return (
      <View style={styles.center}>
        <Text>No se pudo cargar el Pokémon.</Text>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      {pokemon.sprites.front_default && (
        <Image
          source={{ uri: pokemon.sprites.front_default }}
          style={{ width: tamañoImagen, height: tamañoImagen }}
        />
      )}
      <Text style={styles.title}>{pokemon.name}</Text>
      <Text>Tipos: {pokemon.types.map((t) => t.type.name).join(", ")}</Text>
      <Text>Altura: {pokemon.height}</Text>
      <Text>Peso: {pokemon.weight}</Text>
      <View style={styles.stats}>
        {pokemon.stats.map((s) => (
          <Text key={s.stat.name}>
            {s.stat.name}: {s.base_stat}
          </Text>
        ))}
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  container: { flex: 1, alignItems: "center", padding: 24 },
  title: { fontSize: 24, fontWeight: "bold", textTransform: "capitalize", marginVertical: 8 },
  stats: { marginTop: 16, alignItems: "flex-start" },
});
```

```
});
```

```
export default PokemonDetailScreen;
```

Mismo patrón que en `app/index.tsx` : función flecha declarada como constante, y `export default` al final porque este archivo también vive dentro de `app/` y define una ruta (`/pokemon/[name]`).

Con esto ya tienes las dos rutas pedidas funcionando de punta a punta —lista → detalle, con datos reales de una API, tipado y con layout que se adapta al tamaño de pantalla.

9. Verificación final: Plan A y Plan B

La configuración ya la hiciste completa en el módulo 2 — este paso es solo para confirmar que la app terminada corre bien en ambos planes.

Con `npx expo start` corriendo en la raíz del proyecto:

Plan A — Emulador Android 1. Presiona `a` en la terminal (o abre el emulador manualmente desde Android Studio si no estaba prendido). 2. La app se instala y abre sola. Toca un Pokémon de la lista y confirma que navega al detalle. 3. Los cambios en el código recargan solos (Fast Refresh) mientras iteras.

Plan B — Web 1. Presiona `w` en la misma terminal. 2. Prueba achicar y agrandar la ventana del navegador — deberías ver la cantidad de columnas de la lista cambiar (módulo 6.3).

Si ambos flujos funcionan sin tocar una línea de código entre uno y otro, el proyecto está completo.

10. Troubleshooting común

- `adb: command not found` : falta agregar `platform-tools` al `PATH` , o reiniciar la terminal después de configurar `ANDROID_HOME` .
 - **El emulador no aparece al presionar `a`** : ábrelo manualmente primero desde Android Studio y vuelve a presionar `a` .
 - **Metro se queda pegado / cambios no se reflejan**: detén el servidor y corre `npx expo start -c` (limpia caché).
 - **Error de red al hacer `fetch` en el emulador**: verifica que el emulador tenga internet (abre un navegador dentro de él); si más adelante usas un backend local propio, recuerda que `localhost` dentro del emulador no apunta a tu máquina — necesitarías `10.0.2.2` en Android.
 - **Un estilo no se aplica**: revisa que no lo estés poniendo en el `View` padre esperando que "baje" a los hijos — en RN no hay cascada (ver 5.2).
 - **`numColumns` no cambia el layout al redimensionar**: `FlatList` necesita remontarse cuando cambia esa prop; por eso el `key={columnas}` en el ejemplo del módulo 8.1 es importante, no es decorativo.
 - **Errores de tipos que no entiendes**: pásale el mensaje de error tal cual a este chat o a cualquier asistente de TypeScript — casi siempre son un `null` / `undefined` sin manejar, y son justo la razón por la que TypeScript te está ayudando, no estorbando.
-

11. Extras opcionales / siguientes pasos

Cuando tengas la app base funcionando, estas son buenas siguientes prácticas, en orden de dificultad:

1. **Ruta de favoritos** (`app/favoritos.tsx`): guarda IDs de Pokémon favoritos con `useState` en un contexto simple (`React.createContext`), sin persistencia todavía. Esto te da tu tercera ruta.
 2. **Persistencia real** con `@react-native-async-storage/async-storage` para que los favoritos sobrevivan a cerrar la app.
 3. **Buscador**: un `TextInput` que filtre la lista por nombre (aprendes a tipar eventos de texto: `(text: string) => void`).
 4. **Pestañas en vez de stack**: cambia `app/_layout.tsx` a un layout de tabs (`expo-router` trae `Tabs` igual que trae `Stack`).
 5. **Pruebas**: agrega Jest + `@testing-library/react-native` y escribe una prueba simple para el hook `usePokemonList` .
 6. **Publicar**: cuando quieras un build real (no solo Expo Go), investiga EAS Build (`eas build`), ya fuera del alcance de este curso introductorio.
-

12. Cronograma sugerido

Con el contenido ampliado, el curso completo (módulos 1 a 9) ronda entre **7 y 10 horas** de trabajo activo, sin contar el tiempo de descarga de Android Studio en segundo plano. Sugerido en sesiones cortas:

- **Sesión 1** (≈1.5 h): Módulo 1 completo — TypeScript paso a paso, con la autoevaluación al final.
- **Sesión 2** (≈1.5 h, con tiempo muerto de descargas): Módulo 2 — dejar Plan A y Plan B configurados y verificados.
- **Sesión 3** (≈45 min): Módulos 3 y 4 — crear el proyecto, primera prueba de ambos planes, entender las rutas.
- **Sesión 4** (≈1.5 h): Módulo 5 — estilos y Flexbox, la parte que más se practica repitiendo.
- **Sesión 5** (≈1 h): Módulo 6 — diseño responsive, aplicado con ejemplos concretos.
- **Sesión 6** (≈1 h): Módulo 7 — tipos y hooks de la API.
- **Sesión 7** (≈1.5 h): Módulo 8 y 9 — pantallas completas, verificación final en ambos planes.
- **Sesión 8** (abierta): Módulo 11 — uno o dos extras, a tu ritmo.

Si ya tienes Android Studio instalado de antes, te ahorras fácilmente 1-2 horas de la sesión 2.